

CS3301 - Data Structures

Topic: selection sort

1. SUMMARY

Selection sort is a simple sorting algorithm that works by repeatedly finding the minimum element from the unsorted part of the list and swapping it with the first element of the unsorted part. The key principle behind selection sort is to divide the list into two parts: the sorted part and the unsorted part. The algorithm then repeatedly picks the smallest (or largest, depending on the sorting order) element from the unsorted part and swaps it with the first element of the unsorted part.

Selection sort is a stable sorting algorithm, meaning that it preserves the order of equal elements. It has a time complexity of $O(n^2)$ in the worst case, making it less efficient than other sorting algorithms like quicksort or mergesort for large datasets.

The reference mentions that selection sort is used in various applications, including sorting large datasets, sorting arrays, and sorting linked lists. However, it also notes that selection sort has a high time complexity and is not suitable for large datasets.

Types and classifications mentioned in the reference include:

- * Selection sort: a simple sorting algorithm that works by repeatedly finding the minimum element from the unsorted part of the list and swapping it with the first element of the unsorted part.
- * Stable sorting algorithm: a sorting algorithm that preserves the order of equal elements.
- * Time complexity: a measure of the efficiency of an algorithm, with $O(n^2)$ being a high time complexity.

The reference also mentions that selection sort can be applied to various data structures, including arrays and linked lists.

Important points highlighted in the reference include:

- * Selection sort is a simple sorting algorithm that works by repeatedly finding the minimum element from the unsorted part of the list and swapping it with the first element of the unsorted part.
- * Selection sort has a time complexity of $O(n^2)$ in the worst case.
- * Selection sort is a stable sorting algorithm, meaning that it preserves the order of equal elements.

2. SHORT NOTES

Definition and Concept

Selection sort is a simple sorting algorithm that works by repeatedly finding the minimum element from the unsorted part of the list and swapping it with the first element of the unsorted part. The key principle behind selection sort is to divide the list into two parts: the sorted part and the unsorted part. The algorithm then repeatedly picks the smallest (or largest, depending on the sorting order) element from the unsorted part and swaps it with the first element of the unsorted part.

Types and Classifications

Selection sort can be classified as a simple sorting algorithm, a stable sorting algorithm, and a comparison-based sorting algorithm.

Simple Sorting Algorithm

Selection sort is a simple sorting algorithm that works by repeatedly finding the minimum element from the unsorted part of the list and swapping it with the first element of the unsorted part.

Stable Sorting Algorithm

Selection sort is a stable sorting algorithm, meaning that it preserves the order of equal elements.

Comparison-Based Sorting Algorithm

Selection sort is a comparison-based sorting algorithm, meaning that it compares elements in the list to determine the order.

Step by Step Working Principle

The working principle of selection sort can be broken down into the following steps:

1. Start with the first element of the list and compare it with all other elements in the list.
2. Find the smallest (or largest, depending on the sorting order) element in the list.
3. Swap the smallest (or largest) element with the first element of the list.
4. Repeat steps 1-3 until the entire list is sorted.

Important Formulas and Equations

The time complexity of selection sort is $O(n^2)$, where n is the number of elements in the list.

Diagrams Description

A diagram of the selection sort algorithm can be described as follows:

...

```
+-----+
| Unsorted List |
+-----+
| Element 1 | Element 2 | Element 3 | ... | Element n |
+-----+
| (unsorted) | (unsorted) | (unsorted) | ... | (unsorted) |
+-----+
```

```
| Sorted List |  
+-----+  
| Element 1 | Element 2 | Element 3 | ... | Element n |  
+-----+  
| (sorted) | (sorted) | (sorted) | ... | (sorted) |  
+-----+  
...
```

Advantages and Disadvantages

Advantages:

- * Simple to implement
- * Stable sorting algorithm
- * Works well for small datasets

Disadvantages:

- * High time complexity ($O(n^2)$)
- * Not suitable for large datasets

Real World Applications

Selection sort can be applied to various real-world applications, including:

- * Sorting large datasets
- * Sorting arrays
- * Sorting linked lists

Comparison with Related Concepts

Study Material - Generated by AI

Selection sort can be compared with other sorting algorithms, including:

- * Bubble sort: a simple sorting algorithm that works by repeatedly swapping adjacent elements if they are in the wrong order.
- * Quick sort: a fast sorting algorithm that works by selecting a pivot element and partitioning the list around it.
- * Merge sort: a fast sorting algorithm that works by merging two sorted lists into a single sorted list.

Time and Space Complexity

The time complexity of selection sort is $O(n^2)$, where n is the number of elements in the list. The space complexity is $O(1)$, meaning that it uses a constant amount of space.

Important Terminology

The following terminology is important in selection sort:

- * Unsorted list: the list of elements that need to be sorted.
- * Sorted list: the list of elements that are already sorted.
- * Smallest (or largest) element: the element with the smallest (or largest) value in the list.
- * Swap: the process of exchanging two elements in the list.

3. CODING EXAMPLES

C Implementation

```
```c
```

```
#include <stdio.h>
```

```
void selection_sort(int arr[], int n) {
```

```
for (int i = 0; i < n - 1; i++) {
 int min_index = i;
 for (int j = i + 1; j < n; j++) {
 if (arr[j] < arr[min_index]) {
 min_index = j;
 }
 }
 int temp = arr[i];
 arr[i] = arr[min_index];
 arr[min_index] = temp;
}

int main() {
 int arr[] = {5, 2, 8, 1, 9};
 int n = sizeof(arr) / sizeof(arr[0]);
 selection_sort(arr, n);
 for (int i = 0; i < n; i++) {
 printf("%d ", arr[i]);
 }
 return 0;
}
```

### Python Implementation

```
```python
def selection_sort(arr):
    for i in range(len(arr) - 1):
        min_index = i
        for j in range(i + 1, len(arr)):
```

```
    if arr[j] < arr[min_index]:
        min_index = j
    arr[i], arr[min_index] = arr[min_index], arr[i]
return arr
```

```
arr = [5, 2, 8, 1, 9]
print(selection_sort(arr))
'''
```

Algorithm

Step 1: Start with the first element of the list and compare it with all other elements in the list.

Step 2: Find the smallest (or largest, depending on the sorting order) element in the list.

Step 3: Swap the smallest (or largest) element with the first element of the list.

Step 4: Repeat steps 1-3 until the entire list is sorted.

Time and Space Complexity

Best Case: $O(n)$

Average Case: $O(n^2)$

Worst Case: $O(n^2)$

Space Complexity: $O(1)$

4. IMPORTANT QUESTIONS

Q1. (2 marks) What is the time complexity of selection sort?

A1. The time complexity of selection sort is $O(n^2)$.

Q2. (2 marks) What is the space complexity of selection sort?

A2. The space complexity of selection sort is $O(1)$.

Q3. (13 marks) What are the advantages and disadvantages of selection sort?

A3. The advantages of selection sort are that it is simple to implement and stable. The disadvantages are that it has a high time complexity and is not suitable for large datasets.

Q4. (13 marks) What are the real-world applications of selection sort?

A4. Selection sort can be applied to various real-world applications, including sorting large datasets, sorting arrays, and sorting linked lists.

Q5. (13 marks) How does selection sort compare with other sorting algorithms?

A5. Selection sort can be compared with other sorting algorithms, including bubble sort, quick sort, and merge sort.

Q6. (15 marks) What are the important formulas and equations related to selection sort?

A6. The time complexity of selection sort is $O(n^2)$, and the space complexity is $O(1)$.

Q7. (15 marks) What are the important terminology related to selection sort?

A7. The important terminology related to selection sort includes the unsorted list, sorted list, smallest (or largest) element, and swap.

Q8. (15 marks) What are the step-by-step working principle of selection sort?

A8. The step-by-step working principle of selection sort involves finding the smallest (or largest) element in the unsorted part of the list and swapping it with the first element of the unsorted part.

Q9. (2 marks) What is the difference between selection sort and other sorting algorithms?

A9. Selection sort is a simple sorting algorithm that works by repeatedly finding the minimum element from the unsorted part of the list and swapping it with the first element of the unsorted part.

Q10. (2 marks) What are the limitations of selection sort?

A10. The limitations of selection sort are that it has a high time complexity and is not suitable for large datasets.

Q11. (13 marks) What are the advantages of selection sort?

A11. The advantages of selection sort are that it is simple to implement and stable.

Q12. (13 marks) What are the disadvantages of selection sort?

A12. The disadvantages of selection sort are that it has a high time complexity and is not suitable for large datasets.

Q13. (15 marks) What are the real-world applications of selection sort?

A13. Selection sort can be applied to various real-world applications, including sorting large datasets, sorting arrays, and sorting linked lists.

Q14. (15 marks) How does selection sort compare with other sorting algorithms?

A14. Selection sort can be compared with other sorting algorithms, including bubble sort, quick sort, and merge sort.

Q15. (2 marks) What is the time complexity of selection sort?

A15. The time complexity of selection sort is $O(n^2)$.

5. MCQs

Q1. What is the time complexity of selection sort?

- a) $O(n)$
- b) $O(n^2)$
- c) $O(n \log n)$

d) $O(1)$

A1. b) $O(n^2)$

Q2. What is the space complexity of selection sort?

a) $O(1)$

b) $O(n)$

c) $O(n^2)$

d) $O(n \log n)$

A2. a) $O(1)$

Q3. What are the advantages of selection sort?

a) Simple to implement and stable

b) Fast and efficient

c) Suitable for large datasets

d) All of the above

A3. a) Simple to implement and stable

Q4. What are the disadvantages of selection sort?

a) High time complexity and not suitable for large datasets

b) Slow and inefficient

c) Suitable for small datasets

d) All of the above

A4. a) High time complexity and not suitable for large datasets

Q5. What are the real-world applications of selection sort?

a) Sorting large datasets, sorting arrays, and sorting linked lists

b) Sorting small datasets, sorting strings, and sorting dates

- c) Sorting integers, sorting floats, and sorting characters
- d) All of the above

A5. a) Sorting large datasets, sorting arrays, and sorting linked lists

Q6. How does selection sort compare with other sorting algorithms?

- a) Faster than bubble sort and slower than quick sort
- b) Slower than bubble sort and faster than quick sort
- c) Equally fast as bubble sort and slower than quick sort
- d) All of the above

A6. a) Faster than bubble sort and slower than quick sort

Q7. What is the time complexity of selection sort?

- a) $O(n)$
- b) $O(n^2)$
- c) $O(n \log n)$
- d) $O(1)$

A7. b) $O(n^2)$

Q8. What is the space complexity of selection sort?

- a) $O(1)$
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n \log n)$

A8. a) $O(1)$

Q9. What are the important formulas and equations related to selection sort?

- a) Time complexity: $O(n^2)$ and space complexity: $O(1)$

- b) Time complexity: $O(n)$ and space complexity: $O(n)$
- c) Time complexity: $O(n \log n)$ and space complexity: $O(n)$
- d) All of the above

A9. a) Time complexity: $O(n^2)$ and space complexity: $O(1)$

Q10. What are the important terminology related to selection sort?

- a) Unsorted list, sorted list, smallest (or largest) element, and swap
- b) Sorted list, unsorted list, swap, and smallest (or largest) element
- c) Smallest (or largest) element, swap, sorted list, and unsorted list
- d) All of the above

A10. a) Unsorted list, sorted list, smallest (or largest) element, and swap

Q11. What is the step-by-step working principle of selection sort?

- a) Finding the smallest (or largest) element in the unsorted part of the list and swapping it with the first element of the unsorted part
- b) Finding the largest (or smallest) element in the unsorted part of the list and swapping it with the first element of the unsorted part
- c) Finding the middle element in the unsorted part of the list and swapping it with the first element of the unsorted part
- d) All of the above

A11. a) Finding the smallest (or largest) element in the unsorted part of the list and swapping it with the first element of the unsorted part

Q12. What are the limitations of selection sort?

- a) High time complexity and not suitable for large datasets
- b) Slow and inefficient
- c) Suitable for small datasets
- d) All of the above

A12. a) High time complexity and not suitable for large datasets

Q13. What are the advantages of selection sort?

- a) Simple to implement and stable
- b) Fast and efficient
- c) Suitable for large datasets
- d) All of the above

A13. a) Simple to implement and stable

Q14. How does selection sort compare with other sorting algorithms?

- a) Faster than bubble sort and slower than quick sort
- b) Slower than bubble sort and faster than quick sort
- c) Equally fast as bubble sort and slower than quick sort
- d) All of the above

A14. a) Faster than bubble sort and slower than quick sort

Q15. What is the time complexity of selection sort?

- a) $O(n)$
- b) $O(n^2)$
- c) $O(n \log n)$
- d) $O(1)$

A15. b) $O(n^2)$